Ders 3 Genişletilmiş Notu: Kaba Kuvvet, Azalt ve Fethet, Böl ve Fethet Algoritmaları

Ders 3 Genişletilmiş Notu: Kaba Kuvvet, Azalt ve Fethet, Böl ve Fethet Algoritmaları

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Kaba Kuvvet (Brute Force) Algoritmaları
- 2. Alıştırma: Ağ Topolojisini Tanıyan Algoritmalar
- 3. Ayrıntılı Arama (Exhaustive Search)
- 4. Azalt ve Fethet (Decrease and Conquer)
- 5. İkili Arama ve Karşılaştırma Sayısı
- 6. Böl ve Fethet (Divide and Conquer) ve Master Teoremi
- 7. Birleştirmeli Sıralama (Merge Sort)
- 8. Dışbükey Örtü (Convex Hull)
- 9. Medyan Bulma (Selection)
- Örnek Sorular

1. Kaba Kuvvet (Brute Force) Algoritmaları

Geçen hafta temel algoritma sınıfları karmaşıklık açısından ele alınmıştı. Bu derste kaba kuvvet algoritmaları, azalt ve fethet, böl ve fethet algoritmaları, bunların türevleri ve verimlilikleri (efficiency) işleniyor.

Bilgisayar bilimlerinde ilk karşılaşılan örnekler çoğunlukla kaba kuvvet örnekleridir. Bu algoritmalar çalışma gücünü bilgisayarın donanımından alır: bilgisayar ne kadar hızlıysa işlevsellikleri o kadar artar. Problemin en bariz, en göze çarpan çözümü doğrudan koda dökülür. Bu yüzden tasarım maliyetleri çok düşüktür ve küçük veri boyutları için uygundurlar. Veri boyutu büyüdüğünde ise karmaşıklıkları yüzünden uygulanabilir olmaktan çıkarlar ve daha akıllıca çözümler gerekir. Hoca, her seferinde küçük veriyle çalışılacaksa tasarım maliyetini düşürmek için kaba kuvvetin mantıklı bir seçim olduğunu söyledi.

İlk örnek a^n'in hesaplanmasıdır. Standart formülü kullanan kaba kuvvet algoritması $a \cdot a \cdot a \cdots$ diye n kez çarpar; karmaşıklığı $O(n)$'dir. Daha kısa yollar vardır: örneğin a^{16}'yı bulmak için a^8'den yararlanmak. Bu fikir 4. bölümde tekrar ele alınıyor.

Kaba kuvvetin en çok kullanıldığı problemler sıralama, arama, matris çarpımı ve dizgi (string) aramadır:

- **Matris çarpımı:** Geçen derste karmaşıklığı n^3 bulunmuştu; iki matris üzerinde gezilir ve karşılıklı hücrelerin sonucu üçüncü bir matrise yazılır.
- **Bubble sort:** standart hâliyle n^2'dir.
- **Sıralı arama (sequential search):** n elemanlı bir dizide bir anahtar aranır; bulunursa yeri, bulunamazsa -1 döner. Bütün elemanlara tek tek bakıldığı için karmaşıklık n'dir.

1.1 Seçmeli Sıralama (Selection Sort)

Selection sort tipik bir kaba kuvvet sıralamasıdır. Sırasız, n elemanlı bir dizi üzerinde dış döngüyle dönülür. Her turda taranan kısım içinde en küçük eleman bulunur ve turun başındaki elemanla yer değiştirir. İkinci turda dizinin ikinci elemanından başlanır, kalan kısmın en küçüğü bulunup ikinci elemanla yer değiştirir; böyle devam edilir. Slayttaki sözde kod şöyledir (ekrandan, 00:09:15):

```
ALGORITHM SelectionSort(A[0..n-1])
for i <- 0 to n - 2 do
    min <- i
    for j <- i + 1 to n - 1 do
        if A[j] < A[min]  min <- j
    swap A[i] and A[min]
```

Temel işlem karşılaştırmadır. Geçen hafta hatırlatılan toplam formülleriyle:

$$C(n) = \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1 = \sum_{i=0}^{n-2} (n-1-i) = \frac{(n-1)n}{2}$$

Dolayısıyla selection sort Big O notasyonunda n^2'dir. Takas (swap) işlemi hesaplama açısından pahalıdır. Swap'ın kaç kez yapıldığı iç döngüye değil dış döngüye bağlıdır; yani karşılaştırma sayısı n^2 olsa da swap sayısı yalnız n mertebesindedir. Hoca bu yüzden selection sort'un diğer algoritmalara göre tercih edilebilir bir seçenek olduğunu söyledi.

1.2 Kaba Kuvvet Dizgi Eşleme (String Matching)

İki dizgi vardır: n karakterli metin T ve m karakterli desen (pattern) P. T içinde P aranır; bulunursa eşleşmenin başladığı indis, bulunamazsa -1 döner. Slayttaki algoritma (ekrandan, 00:10:00):

```
ALGORITHM BruteForceStringMatch(T[0..n-1], P[0..m-1])
for i <- 0 to n - m do
    j <- 0
    while j < m and P[j] = T[i + j] do
        j <- j + 1
    if j = m return i
return -1
```

Dış döngü $n-1$'e kadar değil, $n-m$'ye kadar gider. Desen metnin sonuna yerleşmiş olabilir, ama sığamayacağı noktadan sonra aramaya gerek yoktur. Taranan her konum için desenin ilk elemanı başlangıç alınarak karşılıklı karşılaştırma yapılır. Karakterler eşit olduğu ve desenin sonu aşılmadığı sürece ilerlenir. Döngüden çıkınca neden çıkıldığına bakılır: j, m'ye ulaştıysa desen bulunmuştur.

Hoca karmaşıklığı öğrencilere sordu; gelen cevaplar arasında n^2 ve $n \cdot m$ vardı. Hocanın açıklaması:

- **En kötü durum:** dışarıda $n-m$'ye kadar giden, içeride m'ye kadar giden bir döngü vardır. En kötü durum, desenin son elemanına kadar sürekli eşleşme yakalanıp son elemanda kaybedilmesidir. Karmaşıklık $O(nm)$.
- **En iyi durum:** ilk bakışta bulunur, 1.
- **Ortalama durum:** dizgi eşlemede genelde ilk birkaç karakterden sonra uygun olmadığı anlaşılır. Uygulandığı alana göre değişse de ortalama durum en kötü durumdan çok en iyi duruma yakındır.

1.3 Hangi Algoritmalar Kaba Kuvvettir?

Hoca geçen haftaki üç algoritmayı hatırlattı: 1'den n'e kadar sayıların karelerini toplayan algoritma, dizideki en büyük ile en küçük eleman farkını bulan algoritma ve matrisin simetrik olup olmadığını kontrol eden algoritma. Bunlar kaba kuvvet midir? Uygulama biçimine göre

daha akıllıca şeyler yapılabilir. Ama örneğin birincisinde 1'den n'e kadar gidip her seferinde kareyi topluyorsak kaba kuvvet algoritması yazmış oluruz. Genel kural: düz (straightforward) bir algoritma, yinelemeler içinde hesaplama ya da karşılaştırma yapıyorsa kaba kuvvettir.

2. Alıştırma: Ağ Topolojisini Tanıyan Algoritmalar

Hoca, yinelemeli algoritmaların analizinin iyice oturması için kâğıt kalemle çalışılacak bir örnek verdi. Üç ağ topolojisi vardır: ring, star ve fully connected mesh. Her biri 0 ve 1'lerden oluşan bir komşuluk matrisiyle verilir: iki düğüm bağlıysa ilgili satır ve sütunun kesiştiği hücrede 1, değilse 0 vardır. Amaç, verilen matrisin bu üç topolojiden hangisine ait olduğunu bulan kaba kuvvet algoritmaları tasarlamak ve karmaşıklıklarını bulmaktır. İki bakış açısı düşünülebilir: sınıflandırma (identification), yani hangi topoloji olduğunu bulmak; ya da doğrulama (verification), yani "bu ring mi, bu star mı" sorusuna cevap vermek.

2.1 Ring

Düğümlere a'dan f'ye harf verilmiştir. A, B ile ve F ile bağlıdır; bu yüzden A satırında B ve F sütunlarında 1 vardır. Graf yönsüzdür, matris simetriktir (ekrandan, 00:20:00):

	a	b	c	d	e	f
a	0	1	0	0	0	1
b	1	0	1	0	0	0
c	0	1	0	1	0	0
d	0	0	1	0	1	0
e	0	0	0	1	0	1
f	1	0	0	0	1	0

Doğrulama için her satırda, bulunulan satırın bir eksiği ve bir fazlası (mod n'e göre) 1, diğerleri 0 olmalıdır. İlk ve son satır, çevrimin kapandığı noktaya denk geldiği için kural dışı gibi görünür; mod alma işlemiyle bu da çözülür. Her satırda yalnız 1 olması gereken yerlere bakılır ve bir sayaçla kaç kez sağlandığı tutulur. Böylece ring, diğer topolojilerden $\Theta(n)$ ile ayırt edilir. Tanıma (identification) için ise bütün matris taranmalıdır; bu n^2 karmaşıklık demektir.

2.2 Star

Star topolojisinde ortadaki düğüm (örnekte c) bütün düğümlere bağlıdır (ekrandan, 00:22:30):

	a	b	c	d	e	f
a	0	0	1	0	0	0
b	0	0	1	0	0	0
c	1	1	0	1	1	1
d	0	0	1	0	0	0
e	0	0	1	0	0	0
f	0	0	1	0	0	0

Merkeze hangi numara verilirse verilsin, bir sütun boyunca ve bir satır boyunca 1'ler görülür; ortadaki cc hücresi 0'dır. Yani matriste "artı" biçiminde bir desen vardır. Doğrulama şöyle yapılabilir: birinci satırda gezilir; ilk 1 görülen yerde o sütun boyunca aşağı inilir, aynı şekilde ilgili satır boyunca kayılır. Bu, tasarıma göre değişse de genel olarak $\Theta(n)$'dir.

2.3 Fully Connected Mesh

Bütün düğümler birbirine bağlıdır: köşegen 0, diğer bütün hücreler 1'dir (ekrandan, 00:23:45). Bu topoloji star ile karışabilir, çünkü içinde star'ları barındırır: star'ı doğrulamak için kullanılan algoritma ek kontrol konmazsa burada da "star" der. Kesin karar için bütün matris hücrelerinin gezilmesi gerekir; karmaşıklık n^2'dir.

3. Ayrıntılı Arama (Exhaustive Search)

Ayrıntılı arama aslında bir kaba kuvvet algoritmasıdır, ama kombinasyon içeren problemler için özelleşmiş bir addır. Faktöriyel karmaşıklığa giden kaba kuvvet algoritmaları bu gruba girer. Önce bütün çözümler, bütün seçenekler ortaya konur; sonra bunlardan gerekli koşulları sağlayan seçilir. Bütün seçenekleri ortaya koymak kombinasyon içeren kısımdır ve karmaşıklık buradan gelir.

Örnek problemler:

- **Sırt çantası (knapsack):** birim ağırlıkları ve değerleri olan eşyalar, toplam değer en büyük olacak şekilde bir çantaya yerleştirilir. Bütün seçenekler tek tek ortaya konup her birinin değeri hesaplanır ve en iyisi seçilir.
- **Atama (assignment):** kişileri işlere atama problemi.
- **Gezgin satıcı (traveling salesman):** bir graf problemi; belirli düğümlerin hepsini gezmek için en verimli yol aranır.

3.1 Atama Problemi

n kişi vardır ve bunlar n işe atanır; her kişiye bir iş verilir. Kişilerin bu işleri yapma maliyetleri bir tabloda verilir ve en düşük toplam maliyetli atama aranır. Tabloda örneğin birinci kişiye birinci iş atanırsa maliyet 9, ikinci iş atanırsa 2, üçüncü iş atanırsa 7'dir.

Çözüm n elemanlı bir dizi olarak gösterilir. Dizideki konum kişiyi, o konumdaki eleman kişinin atandığı işi gösterir. Örneğin $\langle 2, 3, 4, 1 \rangle$: birinci kişi ikinci işe, ikinci kişi üçüncü işe, üçüncü kişi dördüncü işe, dördüncü kişi birinci işe atanmıştır.

Ayrıntılı aramayla çözüm şöyledir: dizi 1'den n'e kadar sayıları içerdiği için bütün seçenekleri listelemek bu sayıların permütasyonlarını bulmak demektir ($\langle 1, 2, 3, 4 \rangle$, $\langle 1, 2, 4, 3 \rangle$, $\langle 1, 3, 2, 4 \rangle$, ...). Her atama seçeneği için maliyet hesaplanır ve en küçük maliyetli olan seçilir. Slayttaki örnek (ekrandan, 00:30:00):

$$C = \begin{bmatrix} 9 & 2 & 7 & 8 \\ 6 & 4 & 3 & 7 \\ 5 & 8 & 1 & 8 \\ 7 & 6 & 9 & 4 \end{bmatrix}$$

Atama	Maliyet
$\langle 1, 2, 3, 4 \rangle$	$9 + 4 + 1 + 4 = 18$
$\langle 1, 2, 4, 3 \rangle$	$9 + 4 + 8 + 9 = 30$
$\langle 1, 3, 2, 4 \rangle$	$9 + 3 + 8 + 4 = 24$
$\langle 1, 3, 4, 2 \rangle$	$9 + 3 + 8 + 6 = 26$
$\langle 1, 4, 2, 3 \rangle$	$9 + 7 + 8 + 9 = 33$
$\langle 1, 4, 3, 2 \rangle$	$9 + 7 + 1 + 6 = 23$

$\langle 1, 2, 3, 4 \rangle$ için: birinci kişiye birinci iş (9), ikinci kişiye ikinci iş (4), üçüncü kişiye üçüncü iş (1), dördüncü kişiye dördüncü iş (4); toplam 18. Slayttaki bütün satırlar python ile hesaplanıp doğrulandı. Slayt "etc." diye devam eder; bütün permütasyonlar yazılmamıştır.

Karmaşıklık: bütün seçenekleri bulmak için permütasyon üretilir, bu $n!$'dir. Bütün maliyetler hesaplandıktan sonra en küçüğü bulmak için bir arama yapılır, bu da n'dir. Genel karmaşıklık yaklaşık $n!$ kabul edilir. Slaytın notu: ayrıntılı arama, çok küçük örnekler dışında pratik değildir.

3.2 Ayrıntılı Aramayla Sıralama

Hoca sordu: sıralama problemi ayrıntılı aramayla nasıl çözülür (ekrandan, 00:36:20)? Dizideki elemanların bütün olası sıralamaları, yani permütasyonları hesaplanır; bu $n!$'lik bir iştir. Her sıralamanın gerçekten sıralı olup olmadığı kontrol edilir; n eleman için bu $n-1$ karşılaştırmadır. Toplam verim $n! \cdot (n-1)$'dir.

Bunun Big O karşılığı nedir? Big O'da $f(n) \leq c\, g(n)$ olacak bir g aranıyordu. Hoca şu eşitsizliği yazdı:

$$n!\,(n-1) < (n+1)!$$

Bu doğrudur, çünkü $(n+1)! = n!\,(n+1)$'dir. Atama probleminde $n!$ seçenek için tek bir toplam vardı ve toplamlar üzerinden bir kez geçiliyordu. Burada ise $n!$ farklı dizilimin her biri için ayrı bir kontrol işletilir. Bu yüzden Big O'da $(n+1)!$ demek doğru olur.

3.3 Hamilton Çevrimi

Bu problem gezgin satıcı probleminin bir alt problemi gibidir. Hamilton yolunda bir kez geçilen kenardan tekrar geçilmez ve yol üzerindeki bütün düğümler birer kez ziyaret edilir. Bir Hamilton yolu bütün düğümleri gezip başladığı yerde bitiyorsa çevrim (circuit) tamamlanmış olur. Slaytta altı düğümlü bir graf ve üzerinde kırmızıyla çizilmiş bir Hamilton çevrimi vardır (ekrandan, 00:36:20).

n düğümlü bir graf için ayrıntılı arama:

- Her düğüm başlangıç kabul edilerek bütün turları içeren permütasyonlar hesaplanır: $n!$.
- Her permütasyonda ardışık düğümler arasında kenar olup olmadığı kontrol edilir; ayrıca son düğüm ile başlangıç düğümü arasında kenar olup olmadığına, yani kapalı bir çevrime dönüp dönmediğine bakılır. Bu, sıralamadaki gibi $n-1$'lik bir kontroldür.
- Sonuç yine $(n+1)!$ ile biten bir karmaşıklıktır.

Hoca bunların daha iyi çözümlerinin ileride görüleceğini, burada amacın ayrıntılı aramayla ne olacağını göstermek olduğunu söyledi.

Soru: Hamilton çevriminde $n!$'lik bir karmaşıklıktan bahsedildi; ama her düğüm her düğüme bağlı değil. $n!$'lik bir dizilim mümkün değil ki? **Cevap:** Ayrıntılı aramanın çalışma prensibi kaba kuvvettir: en basit uygulanabilir hâliyle bütün seçeneklerin listesi dökülür. Algoritma yalnız birbirine bağlı düğümler üzerinden seçenek üretecek şekilde kodlanırsa $n!$'den az seçenek çıkar; ama bunun için ek kontroller gerekir. Altı düğüm için altıya kadar sayıların permütasyonları bilgisayar gücüyle hızla dökülüp sonra bağlantılar kontrol edilebilir. Her iki yaklaşım da sonuca ulaşır. "Fazla seçeneği bilgisayar gücüyle değerlendireyim, uygulama beni uğraştırmasın" yaklaşımı için $n!$ söylendi.

3.4 Kaba Kuvvetin Yeri

Ayrıntılı arama da kaba kuvvet için güzel bir örnektir: uygulaması kolay bir algoritma tasarlanabilir; küçük düğüm sayılarında, küçük dizilerde kısa yoldan sonuca götürür. Hoca bu yüzden bu algoritmaların bir kenara atılamayacağını vurguladı.

> **Hocanın vurgusu:** Önemli olan probleme uygun bir çözüm bulmaktır. Küçük problemde kaba kuvvet yeterlidir; büyük problemde daha akıllı yöntemler gerekir.

4. Azalt ve Fethet (Decrease and Conquer)

Azalt ve fethet tekniğinde, verilen bir problemin bir örneğine ait çözüm ile onun daha küçük bir örneğine ait çözüm arasındaki ilişki ortaya konur. Problemin büyük ve küçük sürümlerinin çözümleri arasındaki ilişkiden yola çıkılarak genel bir çözüm üretilir. Teknik yukarıdan aşağı (top-down) ya da aşağıdan yukarı (bottom-up) uygulanabilir; yukarıdan aşağı uygulamalar özyinelemelidir ve derste bunlarla ilgilenilir.

Üç türü vardır:

- Sabit kadar azaltma (decrease by a constant)
- Sabit çarpanla azaltma (decrease by a constant factor)
- Değişken boyutlu azaltma (variable size decrease)

Hoca bunun böl ve fethetten farklı bir mekanizma olduğunu, onun bir alt türü olmadığını vurguladı.

4.1 Bir Azaltma: a^n

Bir azaltarak ilerlemede problem her adımda bir küçültülür; $n-1$ boyutlu alt probleme çözüm getirilir ve bu çözüm bütüne genelleştirilir. a^n problemi, a^{n-1}'in çözümünün a ile çarpımıyla çözülebilir (ekrandan, 00:40:00):

$$f(n) = \begin{cases} f(n-1) \cdot a & n > 0 \\ 1 & n = 0 \end{cases}$$

Bu özyinelemeli hâldir. Aşağıdan yukarı çözülürse $a \cdot a \cdot a \cdots$ olur, yani standart kaba kuvvete döner. Karmaşıklığı yine n'dir: bu örnekte özyinelemeli yazmak verimlilik kazandırmaz, iş yine n kez tekrarlanır.

4.2 Yarıya Azaltma: a^n

Problem ikiye bölünürse (decrease by half) belirli bir sabit çarpanla küçültüldüğü için sabit çarpanla azaltma grubuna girer. n boyutlu problemin $n/2$ boyutlu kısmı çözülür ve genelleştirilir. a^n, $a^{n/2}$'nin karesidir (ekrandan, 00:42:20):

$$a^n = \begin{cases} (a^{n/2})^2 & n \text{ çift ve pozitif} \\ (a^{(n-1)/2})^2 \cdot a & n \text{ tek} \\ 1 & n = 0 \end{cases}$$

Her seferinde ikiye bölünen yapılar geçen derste görüldüğü gibi logaritmiktir; bu örnek $\Theta(\log n)$'de çözülür.

Bunun böl ve fethetten farkı şudur: böl ve fethette de ikiye bölünerek çalışılır, ama burada veri bölünmez; parça ile bütün arasındaki ilişki kullanılır.

Soru: Sabit çarpanla azaltmanın böl ve fethetten farklı olduğu söylendi; fark tam anlaşılmadı.
Cevap: Böl ve fethette parçalara ayrı işlemler uygulanır ve sonuçlar birleştirilerek (merge) toplam sonuca ulaşılır. Sabit çarpanla azaltmada ise parça-bütün ilişkisi kullanılır: 2^8'i bulmak için 2^4 üzerinden gidilir; 2^9'u bulmak için 2^4 ile 2^4 çarpılır ve bir kez daha 2 ile çarpılır. Parçalara bölüp her biri için kod çalıştırıp sonuçları birleştirmek söz konusu değildir; ilişkinin görünürlüğünden yararlanılır. Böl ve fethette genelde bir dizi varsa o dizi bölünür; matris çarpımında matris dörde bölünür ve her alt parça için problem çalıştırılır. Burada bölme yoktur; matematiksel bir ilişki kullanılıp algoritma ona göre kurulur.

4.3 Değişken Boyutlu Azaltma: Öklid Algoritması

Bu türde her adımdaki azaltma miktarı sabit değildir; yinelemeden yinelemeye değişir. Güzel bir örnek, en büyük ortak böleni bulan Öklid algoritmasıdır:

$$\gcd(u, v) = \gcd(v, u \bmod v)$$

Bu ilişkiden yararlanarak problem küçültüle küçültüle sonuca ulaşılır. Hocanın örneği $\gcd(180, 146)$:

- $180 \bmod 146 = 34$, yani $\gcd(180, 146) = \gcd(146, 34)$.
- $146 \bmod 34 = 10$, yani $\gcd(146, 34) = \gcd(34, 10)$.
- $34 \bmod 10 = 4$, yani $\gcd(34, 10) = \gcd(10, 4)$.
- $10 \bmod 4 = 2$, $4 \bmod 2 = 0$; sonuç $\gcd(180, 146) = 2$ (ara adım eklendi; hoca $\gcd(10, 4)$'te durdu, sonuç python ile doğrulandı).

Her yinelemede değerdeki azalma miktarı farklıdır; bu yüzden değişken boyutlu azaltmaya iyi bir örnektir. Slaytta algoritmanın sözde kodu da vardı.

> **Ek bilgi (derste anlatılmadı):** Öklid algoritmasında özyineleme ikinci parametre 0 olduğunda durur: $\gcd(u, 0) = u$. Yukarıdaki örnekte $\gcd(2, 0) = 2$ ile biter.

5. İkili Arama ve Karşılaştırma Sayısı

5.1 Algoritma

İkili arama (binary search) sabit çarpanla azaltmanın tipik uygulamasıdır. Sıralı bir A dizisinde bir K anahtarı aranır; bulunursa indisi, bulunamazsa -1 döner. Her seferinde dizi ikiye bölünür ve ortadaki indisteki elemanla K karşılaştırılır. K daha büyükse aranan eleman sağ yarıdadır ve arama orada sürer.

Sözde kodun işleyişi şöyledir:

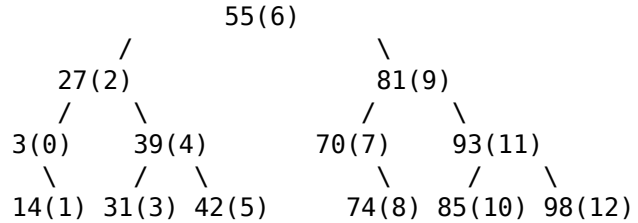
- Aranacak dizi parçasının başını ve sonunu tutan iki değişken vardır: $l = 0$ ve $r = n - 1$; başta bütün diziyi gösterirler.
- Bu ikisi arasında eleman kaldığı sürece ($l \leq r$) bir `while` döngüsü döner. Orta nokta $m = \lfloor (l + r)/2 \rfloor$ olarak bulunur.
- $K = A[m]$ ise eleman bulunmuştur; m döndürülüp çıkılır.
- $K < A[m]$ ise arama sol yarıda sürer: l aynı kalır, $r = m - 1$ olur.
- $K > A[m]$ ise arama sağ yarıda sürer: r aynı kalır, $l = m + 1$ olur.

En iyi durumda eleman ilk atışta bulunur. En kötü durumda bulunamaz ya da sona kadar ikiye bölünerek gidilir; geçen derste görüldüğü gibi karşılaştırma sayısı $\lfloor \log_2 n \rfloor + 1$'dir. Aşağı yuvarlama ve $+1$'in nedeni küçük bir sayı üzerinde analiz yapılınca görülür. Karmaşıklık $\log n$'dir.

5.2 En Fazla Karşılaştırma ve Arama Ağacı

Slayttaki soru (ekrandan, 01:10:00): 3, 14, 27, 31, 39, 42, 55, 70, 74, 81, 85, 93, 98 dizisinde ikili arama en fazla kaç karşılaştırma yapar? Formül bellidir: $n = 13$ için $\lfloor \log_2 13 \rfloor + 1 = 3 + 1 = 4$.

İkinci soru: hangi anahtarlar arandığında en fazla karşılaştırma yapılır? Bir öğrenci en büyük ve en küçük değerleri önerdi. Hoca, en çok uğraştıranın yerleşime bağlı olduğunu, birden fazla eleman olduğunu ve en büyük ile en küçük dışında da aynı karşılaştırma sayısını gerektiren elemanlar bulunduğunu söyledi. Bunu bulmanın kolay yolu diziyi ağaca dökmektir. Ortadaki eleman köke konur: arama ortadan başladığı için kök 55'tir. Sağ yarının ortası (81) kökün sağına, sol yarının ortası (27) soluna konur ve dallanma böyle sürer. Ağaç şöyledir (parantez içinde dizideki indis):



Yapraklara yerleşmiş düğümler en fazla karşılaştırma gerektirenlerdir, çünkü arama kökten o yaprağa giden yolu izler. Örneğin 98 için 55 → 81 → 93 → 98, 74 için 55 → 81 → 70 → 74. Yani 14, 31, 42, 74, 85 ve 98'in her biri 4 karşılaştırmayla bulunur. Bu ağaç farklı veri yapılarında farklı tutulabilir; basit bir dizide tutulursa boş kalan çok sayıda eleman olur, bağlı listeyle de tutulabilir.

> **Ders sonrası not:** Hoca sol çocuğu anlatırken "20" dedi (01:07:52); slayttaki ağaçta bu düğüm 27'dir (ekrandan, 01:10:00). Python ile her anahtar için ikili aramanın karşılaştırma sayısı sayıldı; sonuç slayttaki seviyelerle aynıdır.

5.3 Başarılı Aramada Ortalama Karşılaştırma

Elemanlara ulaşma olasılıkları eşit kabul edilsin. İlk derste ortalama durum verimliliği hesaplanırken dağılım önemliydi ve olasılıklar hesaba katılıyordu. Soru: başarılı bir aramada, yani eleman eninde sonunda bulunacaksa, ortalama kaç anahtar karşılaştırması yapılır? Bir öğrenci $\log n$ dedi; hoca bunun doğru olduğunu, ama önce sayı olarak sormak istediğini söyledi.

Her seviyede kaç düğüm olduğu ve her düğümü bulmak için kaç karşılaştırma gerektiği hesaplanır (ekrandan, 01:12:20):

- Birinci seviye (kök, 55): 1 düğüm, 1 karşılaştırma.
- İkinci seviye (27, 81): 2 düğüm, 2 karşılaştırma.
- Üçüncü seviye (3, 39, 70, 93): 4 düğüm, 3 karşılaştırma.
- Dördüncü seviye, yapraklar (14, 31, 42, 74, 85, 98): 6 düğüm, 4 karşılaştırma. Ağaç dolu olsaydı burada 8 düğüm olurdu.

Toplam, düğüm sayısına bölünür:

$$C_{avg}^{yes} = \frac{1 \cdot 1 + 2 \cdot 2 + 4 \cdot 3 + 6 \cdot 4}{13} = \frac{41}{13} \approx 3{,}15$$

Ders sonrası not: Slaytın açıklama satırında kök "5 (root node, first level)" diye yazılmış (ekrandan, 01:12:20); hoca da "beş" diye okudu. Kök 55'tir. Hesap ($41/13 \approx 3{,}15$) python ile doğrulandı.

Soru: Üçüncü seviyede 4 düğüm varken 3 karşılaştırmayı anlıyorum; ama 6 düğüm varken 4 karşılaştırmayla aranan sayı nasıl bulunur? En az 5 karşılaştırma yapıp, 5. de değilse 6.'dır demek gerekmez mi? **Cevap:** Seviyedeki bütün elemanlarla karşılaştırma yapılmaz; tek bir yol takip edilir. 85 için 55 (1), 81 (2), 93 (3), 85 (4). Öğrenci, seviyedeki bütün elemanlarla kıyaslandığını düşündüğünü söyledi ve konu netleşti.

Formülleştirme: her ağaç seviyesi i için o seviyedeki eleman sayısı 2^{i-1}, o seviyedeki bir düğümü bulmak için gereken karşılaştırma sayısı i'dir. Örneğin $i = 1$ için $1 \cdot 2^0 = 1$, $i = 2$ için $2 \cdot 2^1$. Bu, bütün ağaç dolu olduğunda, yani $n = 2^k$ olduğunda çalışır. Yerleştirmeden önce hangi düğümlerin boş kalacağı bilinemez. Ağaç dengeli olmayabilirdi de: sayılar sıralı verilip ağaca sırayla eklenseydi aşağı doğru uzayan bir yapı oluşurdu. Dengeli ağaç kabulüyle hareket edilince sonuç $\log_2 n$'e varır. Hoca, formül çıkarmak için önce rakam koyup genel bir bakış elde etmenin yararlı olduğunu söyledi.

6. Böl ve Fethet (Divide and Conquer) ve Master Teoremi

6.1 Genel Yapı

Azalt ve fethette problemin bütünüyle alt problemi arasındaki matematiksel ilişki kullanılıyordu; her parça için ayrı kod parçası çalıştırıp birleştirmek, her seferinde özyinelemeli çalışmak gerekmiyordu. Böl ve fethet bu açıdan ayrılır:

- Problem aynı tipte alt problemlere ayrılır; yapılabiliyorsa eşit boyutta, ama bu her zaman mümkün olmayabilir.
- Alt problemler genellikle özyinelemeli olarak çözülür.
- Orijinal probleme ait çözümü elde etmek için sonuçlar birleştirilir (merge).

Örneğin 0'dan $n - 1$'e kadar sayılar ikiye bölünüp yarısı bir fonksiyonda, diğer yarısı aynı fonksiyonda toplanabilir ve sonuçlar birleştirilebilir. Bu mümkündür ama verimli değildir. Böl ve fethet algoritmaları her zaman verimli olmaz; ama işlemler paralel yürütülmek istenirse verimli bir yöne gidilmiş olur. Hoca bunu toplama örneği için söylemediğini belirtti.

6.2 Master Teoremi

Böl ve fethet algoritmalarının karmaşıklığını hesaplamak, standart özyinelemeli algoritmalar kadar kolay olmayabilir. Geçen hafta geriye doğru yerine koyma (backward substitution) ile özyinelemeli algoritmaların karmaşıklığına bakılmıştı; böl ve fethet yapısı gereği daha karmaşık olabilir. Burada master teoremi işe yarar.

Teoremin söylediği şudur: bölmek ya da birleştirmek için kullanılan bir fonksiyon varsa bu, genel karmaşıklığı büyük ölçüde etkiler. Problem her biri n/b boyutlu alt problemlere ayrılıyor ve bunların a tanesi çözülüyorsa (her zaman bütün alt problemleri çözmek gerekmez; bazen bazıları elenir, birinin üzerinden gidilir), çalışma süresi için özyineleme bağıntısı şöyledir (ekrandan, 01:20:00):

$$T(n) = aT(n/b) + f(n)$$

$T(n)$'nin büyüme mertebesi a ve b sabitlerine ve $f(n)$'nin büyüme mertebesine bağlıdır. $f(n) \in \Theta(n^d)$, $d \geq 0$ ise:

$$T(n) \in \begin{cases} \Theta(n^d) & a < b^d \\ \Theta(n^d \log n) & a = b^d \\ \Theta(n^{\log_b a}) & a > b^d \end{cases}$$

Benzer sonuçlar O ve Ω notasyonları için de geçerlidir. Hoca teoremin çok kullanışlı olduğunu ve birçok işi kolaylaştıracağını söyledi.

6.3 Örnek: n Sayının Toplamı

Slayttaki örnek (ekrandan, 01:22:00):

$$a_0 + \cdots + a_{n-1} = (a_0 + \cdots + a_{\lfloor n/2 \rfloor - 1}) + (a_{\lfloor n/2 \rfloor} + \cdots + a_{n-1})$$

$n = 2^k$ kabul edilirse toplama sayısı için bağıntı $A(n) = 2A(n/2) + 1$ olur. İki parçaya bölünür, iki parçayla da işlem yapılır: $a = 2$, $b = 2$. Birleştirmek için sabit zamanda tek bir toplama yapılır: $d = 0$. $a > b^d$ $(2 > 1)$ olduğundan:

$$A(n) \in \Theta(n^{\log_b a}) = \Theta(n^{\log_2 2}) = \Theta(n)$$

7. Birleştirmeli Sıralama (Merge Sort)

7.1 İşleyiş

Merge sort'ta dizi önce parçalara ayrılır: ilk yarısı bir B dizisine, ikinci yarısı bir C dizisine kopyalanır. Fonksiyon kendini özyinelemeli çağırarak bunu eleman sayısı 1'den büyük olduğu sürece tekrarlar. Sonra Merge işlemiyle sıralama tamamlanır. Sort kısmı aslında bölmeyi içerir; işin rengini Merge belirler.

Merge'in sözde kodu: elde iki sıralı dizi vardır, B ve C; bunlar birleştirilerek A elde edilir. Üç indis kullanılır: i, B'yi; j, C'yi tarar; k, A'da sıradaki yerleşim yerini gösterir.

- $B[i] \leq C[j]$ ise (küçükten büyüğe sıralama) $A[k] = B[i]$ yazılır ve i artırılır.
- Değilse, yani C'deki eleman daha küçükse $A[k] = C[j]$ yazılır ve j artırılır.
- Her yerleştirmeden sonra k bir artırılır.
- Dizilerden biri biterse (örneğin $i = p$ olup B'nin elemanları tükenirse) diğerinde kalan elemanlar A'nın sonuna eklenir.

7.2 Örnek

Dizi 8, 3, 2, 9, 7, 1, 5, 4 olsun. İkiye ayrılır: 8, 3, 2, 9 ve 7, 1, 5, 4. Bunlar da ikiye ayrılır ve tek eleman kalana kadar bölmeye devam edilir. Tek elemanlar elde edildikten sonra birleştirme başlar; iki elemanı birleştirmek tek karşılaştırmayla, sabit zamanda yapılır:

```
8 3 2 9 7 1 5 4
8 3 2 9 | 7 1 5 4
8 3 | 2 9 | 7 1 | 5 4
8 | 3 | 2 | 9 | 7 | 1 | 5 | 4
3 8 | 2 9 | 1 7 | 4 5
2 3 8 9 | 1 4 5 7
1 2 3 4 5 7 8 9
```

7.3 Karmaşıklık

Sadelik için n'in ikinin kuvveti olduğu kabul edilir. $C_{merge}(n)$, birleştirme sırasındaki anahtar karşılaştırma sayısıdır (ekrandan, 01:30:00):

$$C(n) = 2C(n/2) + C_{merge}(n), \quad n > 1, \quad C(1) = 0$$

İki yarı için de $n/2$ elemanlı işlem yapılır ve birleştirme için $C_{merge}(n)$ kadar iş eklenir. En kötü durumda birleştirme $n-1$ karşılaştırma ister. Neden? Örneğin 1, 3, 5, 7 ile 2, 4, 6, 8 birleştirilsin. 1 ile 2 karşılaştırılır, küçük olan 1 alınır; 2 ile 3 karşılaştırılır, 2 alınır; 3 ile 4 karşılaştırılır, 3 alınır. Her seferinde sırayla bir diziden, sonra diğerinden eleman alınır; bir dizi erkenden bitip diğeri karşılaştırmasız eklenemez. 8 eleman için 7 karşılaştırma yapılır. Böylece:

$$C_{worst}(n) = 2C_{worst}(n/2) + n - 1, \quad C_{worst}(1) = 0$$

$$C_{worst}(n) = n \log_2 n - n + 1$$

Slayttaki kapalı formül, recurrence ile $n = 2, 4, 8, 16, 1024$ için python'da hesaplanıp doğrulandı.

Master teoremiyle: $a = 2$, $b = 2$; birleştirme $n - 1 \in \Theta(n)$ olduğundan $d = 1$. $a = b^d$ $(2 = 2^1)$ olduğundan $\Theta(n \log n)$.

> **Ders sonrası not:** Hoca derste master teoremini uygularken d için yanlış bir değer yazdı ve "bir yanlışlık var, aradan sonra döneceğim" dedi (01:31:47). Aradan sonra düzeltti (02:17:00): C_{merge}, $n-1$ yani $\Theta(n)$ olduğu için $d = 1$'dir ve sonuç $n \log n$'dir.

8. Dışbükey Örtü (Convex Hull)

8.1 Problem

Düzlemde n nokta verilir. Bu noktalardan, bütün noktaları kapsayan ve dışarıda nokta bırakmayan kapalı bir çevrim oluşturulur; amaç bu koşulu sağlayan en küçük çokgeni bulmaktır. Çözümde kolaylık için bir kabul yapılır: aynı x ya da aynı y koordinatına sahip iki nokta yoktur. Veri yapısı olarak çift yönlü bağlı liste (doubly linked list) kullanılabilir: örneğin $p \to q \to r \to s \to \ldots$ diye kapalı bir çevrim tutulur. Dışbükey örtünün kullanım alanları çoktur: graf teorisi, örüntü tanıma (pattern recognition), coğrafi bilgi sistemleri.

8.2 Kaba Kuvvet Çözümü

Akla ilk gelen, uygulaması en basit çözüm şudur: bütün nokta çiftlerinden doğrular oluşturulur. Bir doğrunun yalnız bir tarafında noktalar varsa, yani noktalar doğrunun iki tarafına yayılmamışsa, bu doğru dışbükey örtü üzerinde bir kenardır.

Matematiksel olarak: (x_1, y_1) ve (x_2, y_2) noktalarından geçen doğrunun denklemi $ax + by = c$ biçimindedir. a, b ve c bu iki noktanın koordinatlarından kurulur. Diğer bütün noktalar denkleme konur. Elde edilen değerlerin işaretleri birbirinden farklıysa noktalar doğrunun farklı taraflarındadır; o doğru örtü üzerinde değildir ve elenir.

Karmaşıklık (ekrandan, 01:40:00): n nokta için $O(n^2)$ olası kenar vardır. Her kenar için sistemdeki n nokta test edilir. Sonuç $O(n^3)$'tür. Slaytta p, q, r, s, t köşeli bir beşgen ve içinde u, v noktaları vardır.

Soru: Noktaların içeride kalıp kalmadığı nasıl kontrol ediliyor? Aynı tarafta olabilirler ama içeride olduklarını garanti edebilir miyiz? **Cevap:** Test edilen şey noktaların içeride kalması değildir. Test edilen şey, bir doğru için diğer bütün noktaların, o doğrunun denklemine konduğunda hep aynı işareti (hep pozitif ya da hep negatif) vermesidir. Örneğin PQ doğrusu için T, U, V, S ve R denenir; hepsi aynı işareti veriyorsa sorun yoktur. Dışarıda, PQ'nun öbür tarafında

bir nokta daha olsaydı aynı işaret alınamazdı. Öğrenci, yalnız içerideki noktaların alındığını sandığını söyledi ve konu netleşti.

Soru: Bu n^3 nasıl oluyor? Geçen derslerde iki karmaşıklık varsa büyük olanın alındığı konuşulmuştu. **Cevap:** Burada iki parça ardışık değil, iç içedir. Kenarları oluşturmak için dönüş sayısı n^2'dir. Örneğin TU doğrusu seçilir, denklemi kurulur; sonra elimdeki bütün noktalar bu denkleme konup pozitif mi negatif mi sonuç ürettiğine bakılır. Bir tarafta diğerleri, Q öbür taraftaysa TU örtünün parçası değildir. Her doğru için n test yapıldığından $n^2 \cdot n = n^3$ olur. TV, UV ya da SU çizildiğinde şekil ikiye bölünür; şekli ikiye bölmemesi gereken kenar bu testle aranır.

8.3 Böl ve Fethet Çözümü

Adımlar:

- Bütün noktalar x koordinatına göre bir kez sıralanır.
- Nokta kümesi iki parçaya ayrılır; her parça için dışbükey örtü hesaplanır ve sonra bunlar birleştirilir.
- Bölme, örtünün sabit zamanda (n'ye bağlı olmadan) bulunabileceği boyuta kadar sürer. Bunun için bir sınır belirlenir: örneğin 5 nokta ya da 3 nokta. 3 noktalı bir parça zaten kendi başına bir dışbükey örtüdür; sonuna kadar gitmek gerekmez.

Hoca bunu bir çizim programında gösterdi. Noktalar x'e göre sıralandıktan sonra x'e göre birbirine en yakın iki nokta, a_1 (sol grubun en sağdaki noktası) ve b_1 (sağ grubun en soldaki noktası) arasından dikey bir L doğrusu çizilir. L'nin solunda ve sağında kalan noktalar ayrı ayrı bölünmeye devam eder. Parçaların eşit sayıda nokta içermesi gerekmez.

Bölmede sorun yoktur; asıl önemli olan birleştirme adımıdır. Bulunan küçük dışbükey örtüler, toplamda doğru bir dışbükey örtü oluşturacak şekilde birleştirilmelidir. Bunun için iki örtüyü birleştiren üst ve alt kenarlar, yani üst teğet (upper tangent) ve alt teğet (lower tangent) bulunur.

8.4 Birleştirme: Kaba Hâli

Şekilde sol örtü A ($a_1, \ldots, a_5$), sağ örtü B (b_1, b_2, b_3) ve aralarında L doğrusu vardır (ekrandan, 01:50:00). A'daki ve B'deki bütün nokta çiftleri için bir doğru oluşturulur ve bu doğrunun L'yi kestiği nokta işaretlenir. a_i ile b_j'yi birleştiren doğrunun L ile kesişim yüksekliğine $y(i, j)$ ya da y_{ij} denir; birinci indis A'daki noktanın, ikinci indis B'deki noktanın numarasıdır. Y ekseninde en yüksek kesişimi veren nokta çifti üst teğettir.

Hoca a_1b_1 ile başladı (y_{11}), sonra a_1b_2 (y_{12}), a_1b_3 (daha aşağıda), sonra a_2 ile b_1, b_2, b_3, sonra a_3 ve a_4 ile denedi. Her adımda "şu ana kadarki en büyükten büyük mü?" diye bakılır; a_4 ile daha büyük bir değer bulundu ve aramaya bütün çiftler tamamlanana kadar devam edildi. Alt teğet için aynı işlem bu sefer y'yi en küçük yapan çift için yürütülür.

Her iki tarafta yaklaşık n nokta varsa bu birleştirme bütün çiftlere baktığı için n^2'dir. Master teoremiyle: iki parçaya bölünür, iki parçayla da ilgilenilir ($a = 2, b = 2$), birleştirme n^2 ($d = 2$). $a < b^d$ ($2 < 4$) olduğundan $T(n) \in \Theta(n^2)$. Hoca bunun analizden de kolayca anlaşılabileceğini söyledi.

8.5 Birleştirme: İyileştirilmiş Hâli

Bütün çiftlere bakmak yerine şu bilgi kullanılır: A ve B dışbükey olduğu için, saat yönünde ve saat yönünün tersinde ilerlerken kesişim yükseklikleri belirli bir düzenle değişir. Algoritma (ekrandan, 02:00:00):

```
1  i = 1
2  j = 1
3  while (y(i, j+1) > y(i, j) or y(i-1, j) > y(i, j))
4      if (y(i, j+1) > y(i, j))      > sağ parmağı saat yönünde ilerlet
5          j = j + 1 (mod q)
6      else
```

```
7          i = i - 1 (mod p)         > sol parmağı saat yönünün tersine ilerlet
8  return (a_i, b_j) as upper tangent
Alt teğet için benzer şekilde.
```

B örtüsünde saat yönünde, A örtüsünde saat yönünün tersinde ilerlenir. Her seferinde birisi sabit tutulup diğeri ilerletilir ve şu ana kadarki en büyük kesişimden daha büyük bir $y(i, j)$ elde edilip edilmediğine bakılır. Örtüler kapalı çevrim olduğu için indisler mod ile güncellenir.

Hocanın izlemesi (ekrandan, 01:50:00 ve 02:00:00):

- $i = 1$, $j = 1$: başlangıç kesişimi y_{11}.
- $y_{12} > y_{11}$ mi? Evet. `or`'un ilk koşulu doğru olduğu için ikincisine bakılmaz; `if`'e girilir, $j = 2$ olur. Yerel en büyük y_{12}.
- $y_{13} > y_{12}$ mi? Hayır. Bu durumda B sabit tutulup A'da hareket edilir: i saat yönünün tersine mod alınınca 5 olur. $y_{52} > y_{12}$ mi? Evet; $i = 5$, yerel en büyük y_{52}.
- A sabit tutulup j artırılırsa daha yüksekten geçilir mi? Hayır. B sabit tutulup i azaltılırsa? Evet; yerel en büyük y_{42} olur.
- Bu noktada A sabitken B'de ilerlemek de, B sabitken A'da ilerlemek de daha büyük bir değer vermez. `while` koşulu sağlanmaz, algoritma biter. Üst teğet (a_4, b_2)'dir.

Böylece üst teğeti bulmak için gereken karmaşıklık n^2'den n'e iner. Master teoremiyle: iki parçaya bölünür, iki parçayla da ilgilenilir ($a = 2$, $b = 2$), birleştirme $\Theta(n^1)$ ($d = 1$). $a = b^d$ olduğundan $\Theta(n \log n)$.

8.6 En Yüksek Noktaları Birleştirmek Yeterli mi?

Hoca sordu: üst sınırı bulmaya çalıştığımıza göre, iki örtünün en yüksek noktalarını birleştirip işi sabit zamanda çözebilir miyiz? Bazen çözülebilir, ama genel olarak çözülemez. Hocanın gösterdiği karşı örnekte iki dışbükey örtü kendi içlerinde doğru işaretlenmiştir. En yüksek noktalar üst sınır kabul edilseydi a_4 ile b_2 birleştirilecekti; ama doğru teğet a_3 ile b_1 arasındadır. Bu yüzden yalnız en üstteki noktalara bakarak üst teğeti, yalnız en alttaki noktalara bakarak alt teğeti bulmak mümkün değildir.

Soru: L doğrusu bir noktanın üzerinden geçiyorsa ne yapılır? **Cevap:** L, A'da x'i en büyük olan nokta (a_1) ile B'de x'i en küçük olan nokta (b_1) arasından geçirilir ve aynı x koordinatına sahip iki nokta olmadığı kabul edilmiştir. Bu yüzden L'nin üzerinde nokta olamaz. Öğrencinin işaret ettiği yerde, örneğin B harfinin hemen altında bir nokta olsaydı, x koordinatı b_1'inkinden küçük olacağı için B örtüsünün bir parçası (örneğin b_4) olurdu; L de bir tık sola kayardı ve arama işlemlerinde o nokta da hesaba katılırdı.

9. Medyan Bulma (Selection)

9.1 Problem ve SELECT Algoritması

Medyan bulma klasik bir problemdir. n elemanlı sırasız bir dizide yalnız medyan değil, verilen sıradaki (rank) eleman aranabilir: beşinci, onuncu, en küçük, en büyük. n eleman varsa $\frac{n+1}{2}$. sıradaki eleman medyandır. Sıralama yaparak bulmak kolaydır: örneğin merge sort ile $n \log n$'de sıralanır, sonra istenen elemana sabit zamanda ulaşılır. Amaç bunu daha hızlı yapmaktır.

Slayttaki algoritma (ekrandan, 02:20:00):

```
SELECT(S, i)
1   Pick x in S          > cleverly
2   Compute k = rank(x)
3   B = { y in S | y < x }
4   C = { y in S | y > x }
5   if k = i
6       return x
7   else if k > i
8       return Select(B, i)
9   else if k < i
10      return Select(C, i - k)
```

Burada i bulunmak istenen sıra, k pivotun sırasıdır. B'de $k-1$ eleman, C'de $h-k$ eleman vardır (h, dizideki eleman sayısı). İşleyiş:

- Diziden bir x elemanı seçilir. Nasıl seçildiği algoritmanın verimliliğini doğrudan etkiler.
- x'in dizideki yeri, yani sırası k olarak düşünülür.
- x'ten küçük elemanlar B'ye, büyük elemanlar C'ye yerleştirilir. Yerleştirme sıralı değildir; her elemana bakılıp büyükse C'ye, küçükse B'ye konur. Bu kolaylık için yapılan bir anlatımdır; kendi veri yapınıza göre düzenleyebilirsiniz.
- $k = i$ ise istenen sayı bulunmuştur, döndürülüp çıkılır.
- Aranan sıra pivotun sırasından küçükse aranan sayı x'in solunda, küçükler arasındadır; SELECT, B ve i ile tekrar çağrılır. B'nin indisleri S'ninkilerle aynı şekilde başladığı için sorun yoktur.
- Aranan sıra daha büyükse arama C üzerinde sürer; ama S üzerindeki değil, C üzerindeki sıra gönderilmelidir: $i - k$.

En iyi durumda tek hamlede bulunur; en kötü durumda dizi sonuna kadar bölünerek gidilir.

9.2 Derste İzlenen Örnek

Hoca bir Word belgesi üzerinde 11 elemanlı bir dizi ile akışı gösterdi (ekrandan, 02:30:00). İndisler 0'dan başlar:

İndis	0	1	2	3	4	5	6	7	8	9	10
S	8	9	5	14	7	13	21	6	12	3	16

- Medyan aranıyor; 0'dan başlandığı için aranan indis $i = 5$. Pivot rastgele seçilebilir; algoritma rastgele seçimde de verimli çalışır. Hoca $x = 7$'yi seçti; dizideki konumu $k = 4$.
- 7'den küçükler sola, büyükler sağa toplandı. Ekrandaki düzen: 5, 3, 6, 7, 8, 13, 21, 9, 14, 12, 16. (Hoca önce 8'i atladı, bir öğrencinin uyarısıyla düzeltti; 12 de sonradan eklendi.)
- $i > k$ ($5 > 4$) olduğu için aranan eleman x'in sağındadır. Hoca önce "B dizisi" dedi, sonra "C dizisi" diye düzeltti. SELECT(C, $5-4$) = SELECT(C, 1) çağrıldı.
- C dizisinde hoca sonlanmanın düzgün görünmesi için iki elemanın yerini değiştirdi ve yeni indisler verdi. Pivot $x = 13$ seçildi; bir öğrencinin "tekrar sıralamayacak mıydık?" uyarısı üzerine elemanlar 13'e göre ayrıldı: solda 8, 9, 12, sonra 13, sağda 14, 21, 16. x'in sırası $k = 3$, $i = 1$; $k > i$ olduğu için SELECT(B, 1) çağrıldı.
- B = 8, 9, 12 üzerinde pivot $x = 9$, sırası $k = 1$. $k = i$; medyan 9 bulundu.

  Ders sonrası not: Hoca 0 tabanlı indis kullanıp k'yı 7'nin dizideki konumu (4) olarak aldı. Slayttaki tanımda $k = rank(x)$ ve B'de $k-1$ eleman vardır, yani sıra 1 tabanlıdır. 7'den küçük 3 eleman (5, 3, 6) olduğundan $rank(7) = 4$, 11 elemanlı dizide medyan için $i = 6$, çağrı SELECT(C, $6-4$) = SELECT(C, 2) olur. 0 tabanlı düşünülürse $rank(7) = 3$ ve doğru çağrı SELECT(C, $i-k-1$) = SELECT(C, 1)'dir; hocanın konumdan aldığı $k = 4$ ile $i-k$ tesadüfen aynı sayıyı verdi. İki yolda da sonuç 9'dur; sıralanmış dizi 3, 5, 6, 7, 8, 9, 12, 13, 14, 16, 21'in ortası python ile doğrulandı.

9.3 Pivot Seçimi ve Karmaşıklık

Pivot, diziyi dengeli bölecek şekilde seçilmelidir; çünkü her seferinde dizinin bir kısmı aramadan çıkarılır, elenir. İyi durumda tek hamlede bulunur. Kötü durum için nasıl bir x seçilmelidir? Bir öğrenci "en küçük ya da en büyük" dedi; hoca doğruladı: her yinelemede en küçük ya da en büyük seçilirse kötü durum oluşur.

Dengeli bölme varsayımı. Master teoremini uygulamak için alt problem boyutu bilinmelidir. x her seferinde diziyi tam ikiye bölüyormuş gibi davranılırsa: özyinelemeli ikiye bölmede genelde $a = 2$ yazılıyordu, ama burada iki parçayla birden ilgilenilmez, biri tamamen elenir; bu yüzden $a = 1$, $b = 2$. Burada birleştirmeden çok bölme karmaşıklığı etkiler: ikiye ayırmak için bütün elemanlar gezilir, yani $f(n) = n$, $d = 1$. $a < b^d$ olduğundan $T(n) \in \Theta(n)$.

Tam ikiye bölünemezse. Bulunan parça yarıya yakın olsun, örneğin her seferinde $7n/10$'luk kısım alınıp özyineleme sürsün. Master teoreminde $a = 1$ (tek alt problem), alt problem boyutu

$n/b = 7n/10$ olduğundan $b = 10/7$, $d = 1$. $b^d = 10/7 > 1 = a$, yine $\Theta(n)$. Tam ikiye bölünemese bile karmaşıklık n'de kalır. Yapılması gereken, olabildiğince dengeli bölen bir yöntem bulmaktır.

En kötü durum. En küçük ya da en büyük seçilirse her seferinde bir eleman ayrılır ve kalan $n-1$ eleman üzerinde özyineleme sürer; bölmek için yine n kadar gezinilir. Hoca bunu master teoremine $a = 1$, "$b = n/(n-1)$", $d = 1$ diye yerleştirmeye çalıştı ve sonucun n olmadığını söyledi. Gerçek çalışma süresi $O(n^2)$'dir (ekrandan, 02:40:00): en küçük eleman için ikiye ayırma n, sonra $n-1$ için ayırma, sonra $n-2$ için ayırma... Dizi küçüldükçe biraz azalır ama toplam n^2 mertebesindedir.

> **Ders sonrası not:** Master teoremi b'nin sabit olmasını ister; $b = n/(n-1)$ sabit olmadığı için teorem bu duruma uygulanamaz. Doğru bağıntı $T(n) = T(n-1) + n$'dir; geriye doğru yerine koyma ile $T(n) = n + (n-1) + \cdots + 1 = \frac{n(n+1)}{2} \in \Theta(n^2)$ çıkar. Hocanın vardığı n^2 sonucu doğrudur.

9.4 Medyanların Medyanı (Median of Medians)

Pivotu dengeli seçmek için şu yöntem kullanılır:

- S dizisi 5 elemanlı sütunlara bölünür. Sütun sayısı $\lceil n/5 \rceil$'tir; n beşe tam bölünmüyorsa sütunlardan biri beşten az eleman içerir.
- Her sütun kendi içinde sıralanır. Sıralamadan kaçınmak için bu iş yapılıyordu diye düşünülebilir; ama her sütunda 5 eleman olduğu için sıralama sabit zamanda yapılır.
- Her sütunun medyanı alınır. Medyanların medyanı bulunur; bunun için aynı SELECT fonksiyonu tekrar çağrılır. Bu değer pivot x olarak kullanılır.

Slayttaki örnekte sütunlar sıralanmış hâlde şöyledir (ekrandan, 02:42:20):

Sütun 1	Sütun 2	Sütun 3	Sütun 4	Sütun 5
1	4	2	2	6
3	5	7	3	**7**
8	**6**	**10**	**4**	17
9	18	11	13	22
15	35	12	70	

Hoca ilk sütunun sıralanmadan önce 1, 8, 9, 3, 15 olduğunu, sıralanınca 1, 3, 8, 9, 15 olduğunu; ikinci sütunun 4, 5, 6, 18, 35'e dönüştüğünü anlattı. Sütun medyanları 8, 6, 10, 4 ve 7'dir. Bunları sıralamak yine $n/5$'e bağlı bir sıralama getirirdi; bunun yerine yazılan fonksiyon kullanılır ve medyanların medyanı 7 bulunur.

> **Ders sonrası not:** Hoca medyanları "8, 6, 14 ve 7" diye okudu; ekranda 8, 6, 10, 4, 7 vardır (ekrandan, 02:42:20). Medyanların medyanı her iki okumada da 7'dir.

7'yi pivot seçmek şunu sağlar: şekilde kırmızıyla gösterilen kısmın (6 dahil) 7'den küçük olduğu, diğer taraftakilerin 7'den büyük olduğu garanti edilir. Böylece dizi büyük ölçüde dengeli ayrılır.

Sonsuz özyineleme sorunu. Hoca sordu: sözde kod, x'i seçmek için yine medyan bulmayı, yani SELECT'i çağırıyor; o da kendi içinde yine pivot seçmek için SELECT'i çağıracak. Bu sonsuz bir döngü oluşturur. Bundan kaçınmak için sözde kod nasıl güncellenmelidir? Cevap: hangi dilde yazılırsa yazılsın, özyinelemeli fonksiyonun çıkışı belli olmalı ve mutlaka karşılaşılacak bir çıkış durumu tanımlanmalıdır. Burada çıkışı belirleyen, eleman sayısının belirli bir makul sabitin (örneğin 5 ya da 10) altına düşmesidir. Bu durumda dizi doğrudan sıralanır, medyanı alınır ve x olarak gönderilir. Böylece sonsuz döngüden kurtulunur.

> **Hocanın vurgusu:** Özyinelemeli bir fonksiyon yazarken çıkış durumu mutlaka karşılaşılacak bir durum olarak belirlenmelidir; yoksa sonsuza giden bir durum oluşur.

9.5 Özyineleme Bağıntısı

Şekilde her sütun bir dikey çizgidir; üstte büyükler, altta küçükler, ortada medyanlar vardır (ekrandan, 02:50:00). Hocanın anlatımında örnek bu kez büyükten küçüğe dizilmişti. $n/5$ sütun yan yana durur; bunların yarısı, yani $n/10$ tanesi x'in bir tarafındadır. Hocanın şekilde gösterdiği iki sütun hariç, bu sütunların her birinde x'ten büyük en az 3 eleman vardır, çünkü medyan sütunun üçüncü elemanıdır. Slayttaki ifade:

$$\text{En az } 3\left(\left\lceil \frac{n}{10} \right\rceil - 2\right) \approx \frac{3n}{10} - 6 \text{ eleman } x\text{'ten büyüktür.}$$

Simetrik olarak aynı sayıda eleman x'ten küçüktür. Bu sayı toplamdan çıkarılınca, özyinelemenin süreceği tarafta en fazla $\frac{7n}{10} + 6$ eleman kalır. Bağıntı:

$$T(n) = \begin{cases} O(1) & n \leq 140 \\ T(\lceil n/5 \rceil) + T(7n/10 + 6) + \Theta(n) & n > 140 \end{cases}$$

- 140, makul bir sabittir (some reasonable constant). Hoca neden bu değerin seçildiğinin sonraki hafta anlatılacağını, belirli bir sınırın üzerinde olmasının yeterli ve çok kritik olmadığını söyledi. Bunun altında iş sabit zamanda yapılır.
- Sınırın üzerinde özyineleme iki yerde çalışır: $T(\lceil n/5 \rceil)$ medyanların medyanını bulmak için (n/5 sütun), $T(7n/10 + 6)$ bir yarıyı eledikten sonra kalan kısım üzerinde devam etmek için.
- $\Theta(n)$, diziyi pivota göre bölmek (ve sütunlara ayırmak) için gerekir.

  Ders sonrası not: Hoca sütun sayısının yarısını "x bölü 10" dedi (02:47:24); doğrusu $n/10$'dur. Altyazıda "7 bölü 16" diye geçen ifade büyük olasılıkla slayttaki $7n/10 + 6$'dır.

Hoca bu bağıntının master teoremiyle ispatlanamayacağını söyledi. Çözümü matematiksel tümevarımla yerine koyma (substitution) yöntemiyle sonraki derse kaldı.

> **Ek bilgi (derste anlatılmadı):** Bu bağıntının çözümü büyük olasılıkla sonraki derste gösterilecektir ve sonucu $T(n) \in O(n)$'dir. Sezgisel nedeni, iki özyinelemeli çağrının toplam boyutunun $n/5 + 7n/10 = 9n/10$, yani n'den sabit bir oranla küçük olmasıdır; her seviyedeki iş geometrik olarak azalır.

Örnek Sorular

Soru 1 (ekrandan, 00:20:00, 00:22:30, 00:23:45). Ring, star ve fully connected mesh topolojileri 0-1 komşuluk matrisleriyle verilmiştir. Verilen matrisin hangi topolojiye ait olduğunu bulan (ya da bir topolojiyi doğrulayan) kaba kuvvet algoritmaları tasarlayın ve karmaşıklıklarını verin.

> **Çözüm (derste verildi):** Ring doğrulama: her satırda yalnız $i - 1$ ve $i + 1$ (mod n) sütunları 1, satır satır kontrol, $\Theta(n)$; tanıma için bütün matris, n^2. Star: merkez satırı ve sütunu boyunca 1'ler; ilk satırdaki ilk 1'den sütuna ve satıra inilerek $\Theta(n)$. Fully connected mesh: köşegen 0, gerisi 1; star algoritması ek kontrol olmadan bunu da kabul eder; kesin karar için bütün matris, n^2.

Soru 2 (derste sözlü, 00:11:45). Kaba kuvvet dizgi eşleme algoritmasının karmaşıklığı nedir?

> **Çözüm (derste verildi):** En kötü durumda (her konumda desenin son karakterine kadar eşleşip son karakterde kaybetme) $O(nm)$; en iyi durumda 1; ortalama durum pratikte en iyi duruma daha yakındır.

Soru 3 (ekrandan, 00:30:00). Maliyet matrisi $C = \begin{bmatrix} 9 & 2 & 7 & 8 \\ 6 & 4 & 3 & 7 \\ 5 & 8 & 1 & 8 \\ 7 & 6 & 9 & 4 \end{bmatrix}$ olan atama problemini ayrıntılı aramayla çözün.

Çözüm (derste verilmedi): Derste ilk altı permütasyon gösterildi (en küçüğü $\langle 1,4,3,2\rangle$: 23). Bütün $4! = 24$ permütasyon python ile denendi. En küçük maliyet $\langle 2,1,3,4\rangle$ atamasıdır: 1. kişi 2. iş (2), 2. kişi 1. iş (6), 3. kişi 3. iş (1), 4. kişi 4. iş (4); toplam $2+6+1+4 = 13$. Sonraki en iyiler $\langle 2,3,1,4\rangle$: 14 ve $\langle 2,4,3,1\rangle$: 17.

Soru 4 (ekrandan, 00:36:20). Ayrıntılı arama sıralama problemine nasıl uygulanır ve karmaşıklığı nedir?

Çözüm (derste verildi): Bütün $n!$ permütasyon üretilir, her biri için $n-1$ karşılaştırmayla sıralı olup olmadığı kontrol edilir. $n!\,(n-1) < (n+1)!$ olduğundan $O((n+1)!)$.

Soru 5 (ekrandan, 00:36:20). Ayrıntılı arama Hamilton çevrimi problemine nasıl uygulanır?

Çözüm (derste verildi): Düğümlerin bütün permütasyonları ($n!$) üretilir; her permütasyonda ardışık düğümler arasında ve son düğümle ilk düğüm arasında kenar olup olmadığı kontrol edilir ($n-1$). Karmaşıklık yine $(n+1)!$ ile biter.

Soru 6 (ekrandan, 01:10:00). 3, 14, 27, 31, 39, 42, 55, 70, 74, 81, 85, 93, 98 dizisinde bir anahtarı ararken ikili arama en fazla kaç anahtar karşılaştırması yapar? Bu sayıda karşılaştırma gerektiren bütün anahtarları listeleyin.

Çözüm (derste verildi): $\lfloor \log_2 13 \rfloor + 1 = 4$. İkili arama ağacının yaprakları en fazla karşılaştırmayı gerektirir: 14, 31, 42, 74, 85, 98.

Soru 7 (ekrandan, 01:12:20). Aynı dizide, bütün anahtarlar eşit olasılıklı ise başarılı bir aramada ortalama kaç anahtar karşılaştırması yapılır?

Çözüm (derste verildi): $C_{avg}^{yes} = \frac{1\cdot1+2\cdot2+4\cdot3+6\cdot4}{13} = \frac{41}{13} \approx 3{,}15$.

Soru 8 (derste sözlü, 01:51:33). Dışbükey örtüde A ve B örtülerinin her birinde n nokta varsa, bütün çiftlere bakan birleştirmenin maliyeti nedir? Böl ve fethet algoritmasının toplam karmaşıklığı ne olur?

Çözüm (derste verildi): Bütün nokta çiftleri için L ile kesişim hesaplanıp en büyüğü arandığı için birleştirme n^2'dir. $T(n) = 2T(n/2) + n^2$: $a = 2 < b^d = 4$, $\Theta(n^2)$. İyileştirilmiş teğet aramasıyla birleştirme n'e iner ve toplam $\Theta(n \log n)$ olur.

Soru 9 (derste sözlü, 02:02:41). Üst teğeti bulmak için iki örtünün en yüksek noktalarını birleştirmek yeterli midir?

Çözüm (derste verildi): Hayır. Hocanın karşı örneğinde en yüksek noktalar a_4 ve b_2 iken doğru üst teğet a_3-b_1'dir. Yalnız en üst ya da en alt noktalara bakarak teğet bulunamaz.

Soru 10 (derste sözlü, 02:34:35). SELECT algoritmasında pivot x nasıl seçilmelidir? En iyi ve en kötü durumda ne olur?

Çözüm (derste verildi): Pivot diziyi dengeli bölmelidir. En iyi durumda ilk pivot aranan elemandır, tek hamlede bulunur. Pivot her seferinde en küçük ya da en büyük seçilirse en kötü durum oluşur ve çalışma süresi $O(n^2)$ olur. Dengeli bölmede (tam yarı ya da $7n/10$ gibi sabit oran) $\Theta(n)$'dir.

Soru 11 (derste sözlü, 02:43:30). Medyanların medyanı yönteminde pivotu seçmek için SELECT tekrar çağrılır ve bu sonsuz bir özyineleme oluşturur. Sözde kod nasıl güncellenmelidir?

Çözüm (derste verildi): Bir çıkış koşulu eklenir: dizinin eleman sayısı makul bir sabitin (örneğin 5, 10; slaytta 140) altındaysa dizi doğrudan sıralanır ve medyanı döndürülür. Böylece özyineleme mutlaka sonlanır.

Dersin Toparlaması

Bu derste algoritma tasarım teknikleri en basitinden başlanarak ele alındı. Kaba kuvvet, problemin en bariz çözümünü bilgisayar gücüne dayanarak uygular; selection sort, dizgi eşleme ve topoloji tanıma örneklerinde karmaşıklıkları hesaplandı. Kombinatorik problemlerde kaba kuvvet ayrıntılı arama adını alır ve atama, sıralama ve Hamilton çevrimi örneklerinde $n!$ ya da $(n+1)!$ mertebesine çıkar. Azalt ve fethet, büyük problemin çözümünü küçük örneğinin

çözümüne bağlayan matematiksel ilişkiyi kullanır: bir azaltma, sabit çarpanla azaltma (yarıya azaltma, ikili arama) ve değişken boyutlu azaltma (Öklid). İkili arama üzerinde en kötü ve ortalama karşılaştırma sayıları bir arama ağacıyla bulundu.

Böl ve fethet ise veriyi gerçekten böler, parçaları ayrı çözer ve birleştirir; karmaşıklığı master teoremiyle hesaplanır. n sayının toplamı $\Theta(n)$, merge sort $\Theta(n \log n)$ bulundu. Dışbükey örtüde kaba kuvvet $O(n^3)$, böl ve fethet ise birleştirme adımının nasıl yapıldığına göre $\Theta(n^2)$ ya da $\Theta(n \log n)$ verir. Son olarak medyan bulmada pivot seçiminin önemi ve medyanların medyanı yöntemi görüldü. Bu yöntemin özyineleme bağıntısı master teoremine uymadığı için matematiksel tümevarımla yerine koyma yöntemiyle çözümü sonraki derse bırakıldı.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.